

# CS201 – Data Structures II

## Lecture 01

Dr. Muhammad Mobeen Movania

# Outline

- Introduction to Data Structures
- Why do we need efficiency?
- Abstract Data Types (ADTs)
- Why use ADTs?
- ADT examples
  - Queue/Stack/Deque/List/USet/SSet interfaces

# Data Structures

- Conceptual and concrete ways to organize data.
  - Required for efficient storage and efficient manipulation
  - Used in designing of efficient algorithms
- Usually abstracted into ADTs or abstract data types.
  - We know **what** a data type does but **how** it does it is hidden.

# Why do we need efficiency?

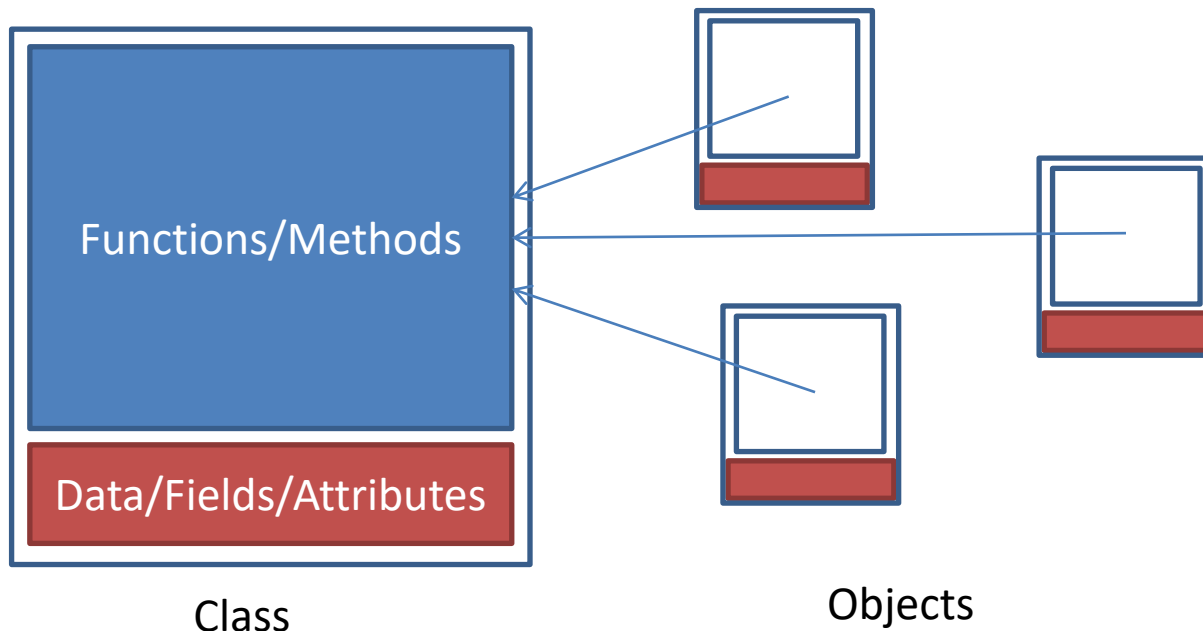
- Example
  - We have an application which processes 1 million items ( $10^6$ )
  - Using linear search each item will be visited at least once making total accesses ( $10^{12}$ )
  - Today's processor can do one billion ( $10^9$ ) operations per second
  - It will take 1000 seconds to process this data which is approx. 16min 40sec

# Why do we need efficiency?

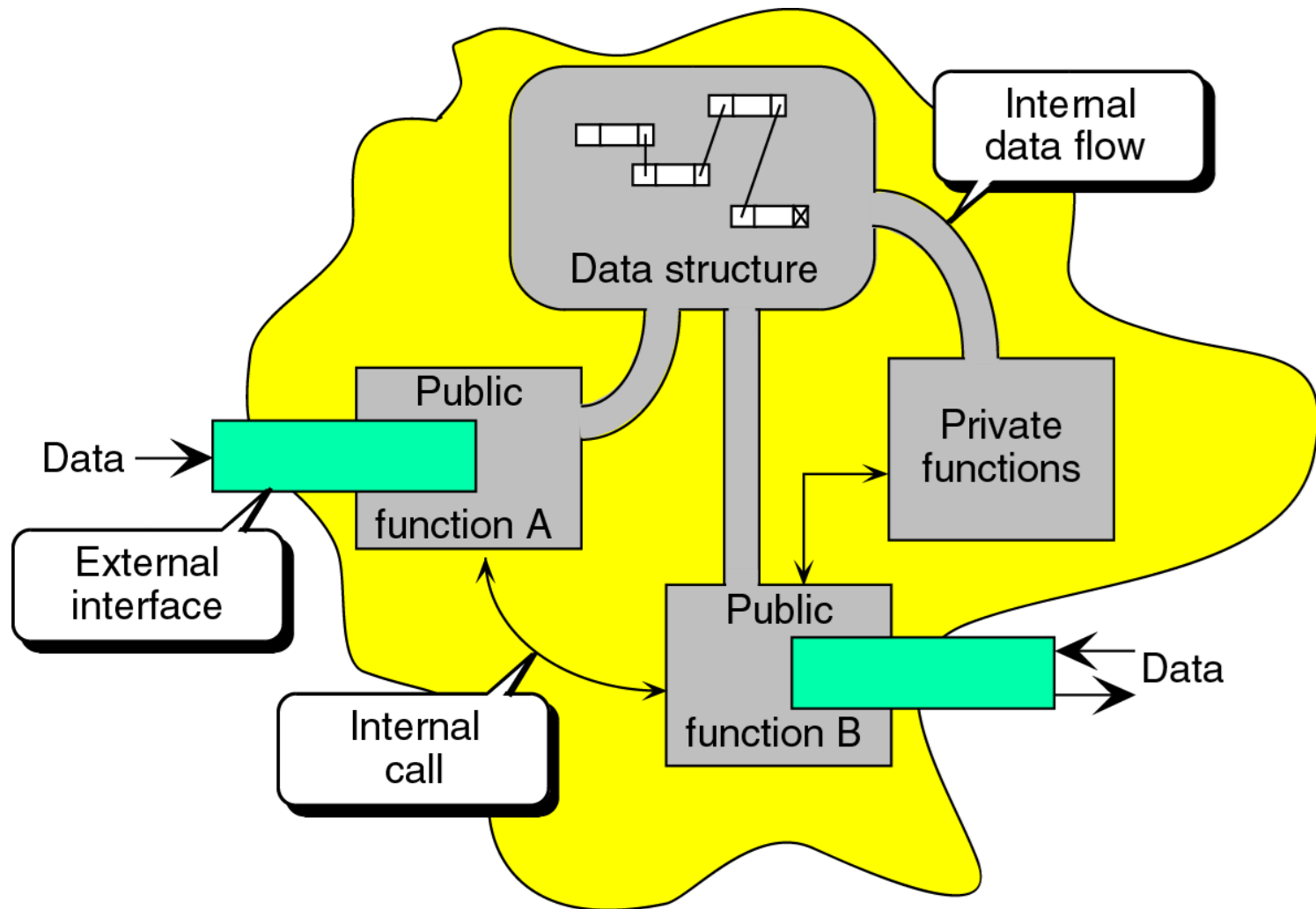
- Example
  - Google has 8.5 billion pages indexed
  - With today's processors doing one billion ( $10^9$ ) operations per second, it will take 8.5 seconds to process this data
- **but**
  - We notice that the Google query results are given in less than this time
  - How is that possible?
    - By using efficient data structures and algorithms so choice of the data structure used is extremely important

# The Abstract Data Type (ADT)

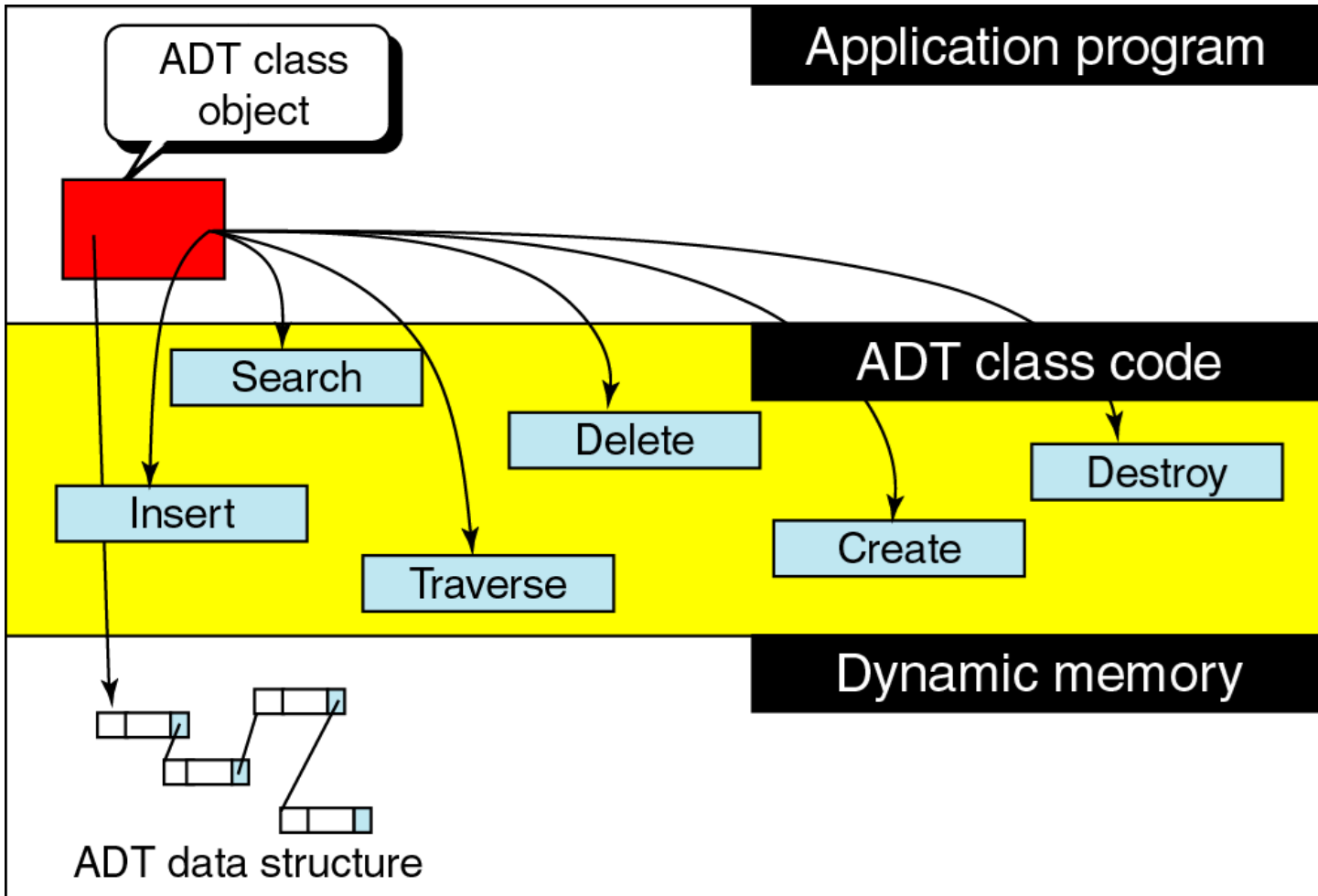
- Represented in a class
- Contains both data and the methods that act on the data



# ADT Class Interface



# Data structure hidden in ADT definition





# Why do we use ADTs?

- ADT represents the interface to the data structure or type
- Interface tells what needs to be done
- Mechanism tells how it will be done
- Its important to separate the interface from the mechanism/implementation
  - Why?
    - It helps update the definition while keeping the same interface so callers of the ADT can get a new (improved) implementation but using the same interface

# Example of ADT

- List interface which can be used to model a data container with multiple items
  - can be implemented using an array based or a linked list (pointer) based implementation
- Array based implementation is
  - used when we have very few insertions/deletions or a lot of direct accesses to data elements
- Linked list based implementation is
  - used when we have a lot of insertions/deletions

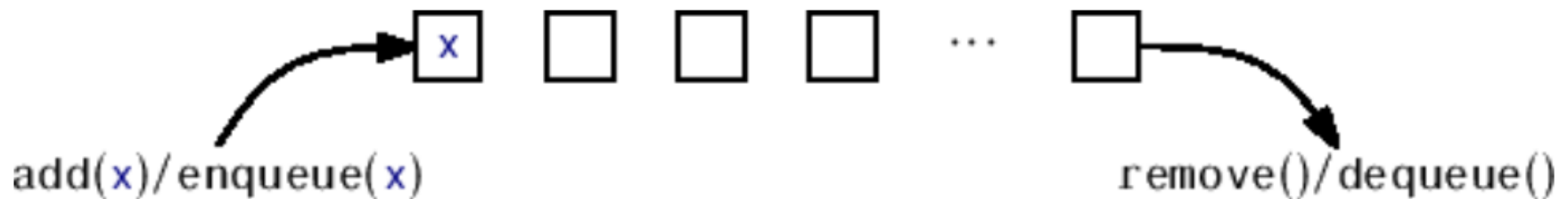
# ADT example:

## Queue/Stack/Deque interface

- Queue:
  - A collection/container that can **add** or **remove** elements
- Supported Operations
  - add(x): adds x to the container
  - remove(): removes y from container and returns y to the caller
- Three queuing disciplines
  - FIFO
  - LIFO
  - Priority

# FIFO Queue

- Removes items in the same order they were added in the container
- Supported Operations:
  - `add(x)` aka `enqueue(x)` adds `x` at back/rear end
  - `remove()` aka `dequeue()` removes `y` from front end



**Figure 1.1:** A FIFO Queue.

# LIFO Queue

- Removes the most recent item from the queue.
- Usually called a **Stack**
- Supported Operations:
  - add(x) is called push(x)
  - remove() is called pop()

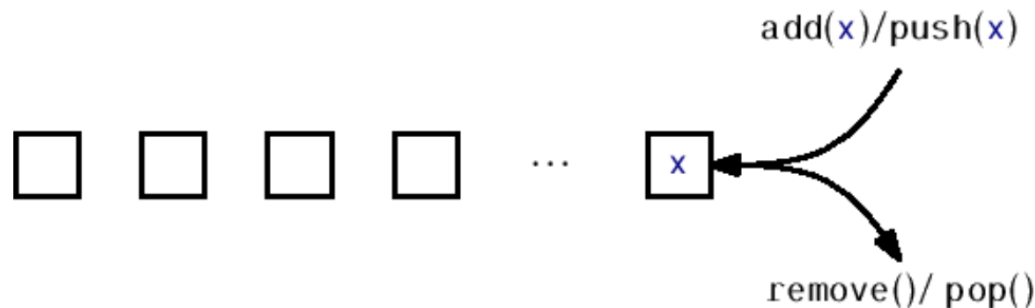


Figure 1.3: A stack.

# Priority Queue

- Removes the smallest element from the queue.
- Supported Operations:
  - `add(x)`
  - `remove()/deleteMin()` removes the minimum of all the elements currently in the queue

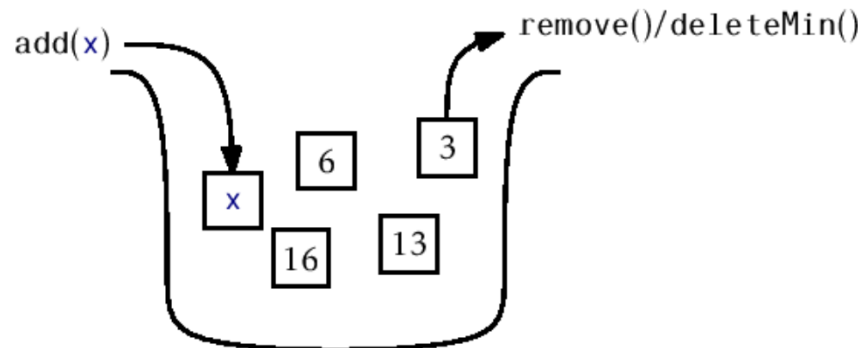


Figure 1.2: A priority Queue .

# Deque Interface

- Generalisation of a LIFO stack and a FIFO queue.
- Represents a sequence of elements, with two ends: a front and a back.
- Elements can be added or removed from both ends

# Deque Interface

- Operations:
  - `addFirst(x)` - adds items at the front of the container
  - `removeFirst()` - removes items from the front of the container
  - `addLast(x)` - adds items at the back of the container
  - `removeLast()` - removes items from the back of the container
- LIFO queue (Stack) can be implemented using `addFirst(x)` and `removeFirst()`
- FIFO queue can be implemented using `addLast(x)` and `removeFirst()`



# List Interface

- A more generic interface to represent FIFO, LIFO queues and deque.
- Operations:
  1. `size()`: return  $n$ , the length of the list
  2. `get(i)`: return the value  $x_i$
  3. `set(i,x)`: set the value of  $x_i$  equal to  $x$
  4. `add(i,x)`: add  $x$  at position  $i$ , displacing  $x_1, \dots, x_{n-1}$  ;  
Set  $x_{j+1} = x_j$ , for all  $j \in \{n-1, \dots, i\}$ , increment  $n$ , and set  $x_i = x$
  5. `remove(i)` remove the value  $x_i$ , displacing  $x_{i+1}, \dots, x_{n-1}$  ;  
Set  $x_j = x_{j+1}$ , for all  $j \in \{i, \dots, n-2\}$  and decrement  $n$

# Queue using the List interface

| Queue interface function | List interface function |
|--------------------------|-------------------------|
| enqueue(x)               | add(size(),x)           |
| dequeue()                | remove(0)               |

# Stack using the List interface

| Stack interface function | List interface function |
|--------------------------|-------------------------|
| push(x)                  | add(size(),x)           |
| pop()                    | remove(size()-1)        |
| top()                    | get(size()-1)           |

# Deque using the List interface

| Deque interface function | List interface function |
|--------------------------|-------------------------|
| addFirst(x)              | add(0,x)                |
| removeFirst()            | remove(0)               |
| addLast(x)               | add(size(), x)          |
| removeLast()             | remove(size()-1)        |

# USet Interface

- An unordered set of **unique** elements which mimics mathematical sets.
- Operations:
  1. `size()`: return the number, `n`, of elements in the set
  2. `add(x)`: add the element `x` to the set if not already present;  
Add `x` to the set provided that there is no element `y` in the set such that `x` equals `y`. Return `true` if `x` was added to the set and `false` otherwise.
  3. `remove(x)`: remove `x` from the set;  
Find an element `y` in the set such that `x` equals `y` and remove `y`. Return `y`, or `null` if no such element exists.
  4. `find(x)`: find `x` in the set if it exists;  
Find an element `y` in the set such that `y` equals `x`. Return `y`, or `null` if no such element exists.

# SSet Interface

- A sorted set interface that stores elements with some sorted order.
- Provides a function `compare(x,y)` as follows:

$$\text{compare}(x,y) \begin{cases} < 0 & \text{if } x < y \\ > 0 & \text{if } x > y \\ = 0 & \text{if } x = y \end{cases}$$

An `SSet` supports the `size()`, `add(x)`, and `remove(x)` methods with exactly the same semantics as in the `USet` interface. The difference between a `USet` and an `SSet` is in the `find(x)` method:

1. `find(x)`: locate `x` in the sorted set;

Find the smallest element `y` in the set such that `y ≥ x`. Return `y` or `null` if no such element exists.

# Difference between USet and SSet Interfaces

- SSet::find(x) operation is called **successor search**.
- It differs in a fundamental way from USet::find(x) since it returns a **meaningful result** even when there is no element equal to x in the set.
  - It returns the first value  $\geq$  to the target value or null if no such value exists
- This requires higher implementation complexity and larger running time

# Book Readings

- ODS: Chapter 1 pages: 1-9



# Next Lecture

- Lecture 02